Rockchip USB SQ Tool Introduction

ID: RK-SM-YF-195

Release Version: V2.0.0

Release Date: 2026-06-23

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, Software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

Since the USB PHY manual for Rockchip platforms has not been publicly released, and the chip TRM lacks detailed USB PHY register descriptions, developers have no effective software means to debug USB signal quality issues. To address this, Rockchip has released the USB SQ (Signal Quality) Tool to simplify USB PHY signal quality debugging. This tool supports most Rockchip chips and provides functions to query USB PHY signal compliance test commands, query signal quality adjustment commands, and automatically generate the corresponding PHY driver code based on the adjustment results.

Product Version

Chip Models	Kernel Version
All Rockchip Chips	Linux-4.4 and above RT-Thread-4.1 and above

Intended Audience

This document (this guide) is primarily intended for the following engineers:

- Hardware Engineers
- Software Engineers
- Technical Support Engineers

Revision History

Date	Version	Author	Modification Description
2021-09-16	V0.0.1	Zheng Jianwei	Initial version
2022-01-10	V1.0.0	Zheng Jianwei	Added platform descriptions for RK3588, RK3588S, RK3326S, PX30S, RK3308, RK3228, and RK3328
2022-04-18	V1.1.0	Zheng Jianwei	Added platform descriptions for RV1103 and RV1106
2022-05-06	V1.2.0	Wu Liangfeng	Added platform descriptions for RV1103 and RV1106, updated RK3588 USB 3.1 Tuning description, and revised formatting
2022-06-25	V1.3.0	Zheng Jianwei	Added platform descriptions for RV1109, RV1126, RK3036, and RK312X
2023-01-15	V1.4.0	Zheng Jianwei	Added platform descriptions for RK3528 and RK3562, and added RK3566 and RK3568 USB 3.1 Tuning descriptions
2023-07-18	V1.5.0	Zheng Jianwei	Added platform description for RK1808 and added RK3399 USB 3.1 Tuning description
2024-04-16	V1.6.0	Zheng Jianwei	Added platform descriptions for RK3576 and RK2118
2024-08-07	V1.7.0	Zheng Jianwei	Added platform descriptions for RK3506 and RV1103B
2025-08-08	V1.8.0	Zheng Jianwei	Added platform descriptions for RK182X, RK2116, RV1126B, and RV1126B-P
2026-03-23	V1.9.0	Zheng Jianwei	Added platform descriptions for RK3538 and RK3572
2026-06-23	V2.0.0	Zheng Jianwei	Modified document format according to Rockchip documentation writing specifications Corrected typos and technical errors

Contents

Rockchip USB SQ Tool Introduction

1. Rockchip USB PHY Signal Adjustment Guidelines
2. Rockchip USB SQ Tool Interface Description
    - 2.1 Rockchip USB SQ Tool Main Interface Description
    - 2.2 RK1808 USB2.0 Tuning Interface Description
    - 2.3 RK3399 USB2.0 Tuning Interface Description
    - 2.4 RK3568 USB2.0 Tuning Interface Description
    - 2.5 RK3588 USB2.0 Tuning Interface Description
    - 2.6 RV1106 USB2.0 Tuning Interface Description
    - 2.7 RV1109 USB2.0 Tuning Interface Description
    - 2.8 RK1808 USB 3.1 Tuning Interface Description
    - 2.9 RK182X USB 3.1 Tuning Interface Description
    - 2.10 RK3399 USB 3.1 Tuning Interface Description
    - 2.11 RK3528 USB 3.1 Tuning Interface Description
    - 2.12 RK3588 USB 3.1 Tuning Interface Description
3. Rockchip USB PHY Tuning Code Generation Instructions
    - 3.1 RK1808 USB2.0 PHY
    - 3.2 RK182X USB2.0 PHY
    - 3.3 RK2116 USB2.0 PHY
    - 3.4 RK2118 USB2.0 PHY
    - 3.5 RK3036/RK312X USB2.0 PHY
    - 3.6 RK3228 USB2.0 PHY
    - 3.7 RK3288 USB2.0 PHY
    - 3.8 RK3308B-S/RK3308 USB2.0 PHY
    - 3.9 RK3326S/PX30S/RK3326/PX30/RK3328 USB2.0 PHY
    - 3.10 RK3399 USB2.0 PHY
    - 3.11 RK3506 USB2.0 PHY
    - 3.12 RK3528 USB2.0 PHY
    - 3.13 RK3538 USB2.0 PHY
    - 3.14 RK3562 USB2.0 PHY
    - 3.15 RK3568/RK3566 USB2.0 PHY
    - 3.16 RK3572 USB2.0 PHY
    - 3.17 RK3576 USB2.0 PHY
    - 3.18 RK3588 USB2.0 PHY
    - 3.19 RV1103/RV1106 USB2.0 PHY
    - 3.20 RV1103B USB2.0 PHY
    - 3.21 RV1109/RV1126 USB2.0 PHY
    - 3.22 RV1126B/RV1126B-P USB2.0 PHY
    - 3.23 RK1808 USB 3.1 PHY
    - 3.24 RK182X USB 3.1 PHY
    - 3.25 RK3399 USB 3.1 PHY
    - 3.26 RK3528/RK3562/RK3566/RK3568/RK3572 USB 3.1 PHY
    - 3.27 RK3576 USB 3.1 PHY
    - 3.28 RK3588 USB 3.1 PHY
    - 3.29 RV1126B USB 3.1 PHY

1. Rockchip USB PHY Signal Adjustment Guidelines

When signal quality issues occur on the USB module in practical applications, they can be resolved by optimizing the hardware circuit or by software tuning of the USB PHY registers. Regardless of which method is used, it is recommended to first test the USB Eye Diagram and perform signal analysis and debugging based on the test report. The Rockchip USB SQ Tool described in this document is a software optimization method that can effectively improve the following common USB issues:

- USB Eye Diagram specification test failures.
- USB enumeration failures (e.g., caused by signal quality issues or PHY supply voltage differential problems).
- Automatic abnormal disconnection of connected USB peripherals.
- Failure to detect disconnection events when USB peripherals are unplugged.

Note: Since USB signal-related issues are highly dependent on the hardware environment (e.g., poor cable quality, excessive cable length, or excessive impedance in traces and cables), hardware environment optimization should be the primary consideration when such problems are encountered. Software signal adjustment should only be considered when the hardware environment cannot be further optimized.

2. Rockchip USB SQ Tool Interface Description

2.1 Rockchip USB SQ Tool Main Interface Description

The USB SQ Tool currently supports USB2.0 tuning on the following platforms: RK1808, RK182X, RK2116, RK2118, RK3036, RK312X, RK3228, RK3288, RK3308, RK3308B-S, RK3326, RK3326S, PX30, PX30S, RK3399, RK3506, RK3528, RK3538, RK3562, RK3566, RK3568, RK3572, RK3576, RK3588, RK3588S, RV1103, RV1103B, RV1106, RV1106B, RV1109, RV1126, RV1126B, RV1126B-P.

Note: For RK3328, only USB2.0 OTG and USB2.0 HOST ports are currently supported, U2PHY for USB 3.0 is not yet supported.

USB 3.1 tuning is supported on: RK1808, RK182X, RK3399, RK3528, RK3562, RK3566, RK3568, RK3572, RK3576, RK3588, and RV1126B.

Support for additional platforms will be rolled out sequentially following development and validation. This tool currently runs only on the Windows platform.

The download path for the Rockchip USB SQ Tool is: https://redmine.rock-chips.com/documents/113

Double-click the Rockchip USB SQ Tool.exe to launch the tool. The platform selection interface is displayed first, as shown in Figure 1. Click the button for the desired platform to enter its adjustment interface.

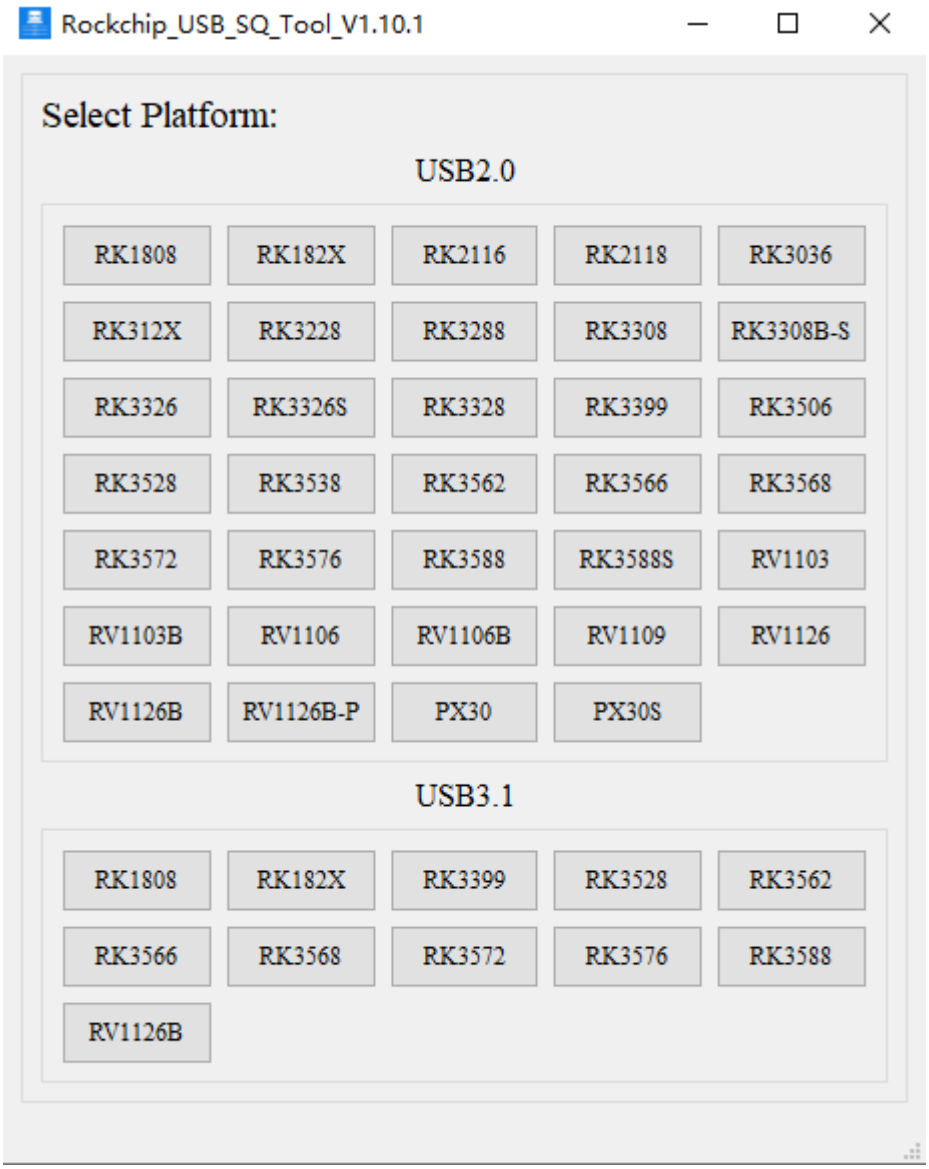


Figure 1 USB SQ Tool Main Window

The PHY Tuning interface remains consistent across various platforms, therefore, a single platform is selected for illustrative purposes. Table 1 provides an example of the USB2.0 PHY Tuning interface, while Table 2 provides an example of the USB 3.1 PHY Tuning interface.

Table 1 Example of USB2.0 PHY Tuning Interface

Platform	USB2.0 PHY Tuning Interface Example
RK1808	RK1808 USB2.0 Tuning Interface Description
RK3036, RK312X, RK3228, RK3308, RK3326, RK3328, RK3399, PX30	RK3399 USB2.0 Tuning Interface Description
RK182X, RK3308B-S, RK3326S, RK3506, RK3528, RK3538, RK3562, RK3566, RK3568, RV1126B, RV1126B-P, PX30S	RK3568 USB2.0 Tuning Interface Description
RK3572, RK3576, RK3588, RK3588S	RK3588 USB2.0 Tuning Interface Description
RK2116, RK2118, RV1103, RV1103B, RV1106, RV1106B	RV1106 USB2.0 Tuning Interface Description
RV1109, RV1126	RV1109 USB2.0 Tuning Interface Description

Table 2 Example of USB 3.1 PHY Tuning Interface

Platform	USB 3.1 PHY Tuning Interface Example
RK1808	RK1808 USB 3.1 Tuning Interface Description
RK182X	RK182X USB 3.1 Tuning Interface Description
RK3399	RK3399 USB 3.1 Tuning Interface Description
RK3528, RK3562, RK3566, RK3568, RK3572, RV1126B	RK3528 USB 3.1 Tuning Interface Description
RK3576, RK3588, RK3588S	RK3588 USB 3.1 Tuning Interface Description

Note: PHY Tuning must be performed strictly in the sequence presented on the tool interface. Progression to subsequent adjustment steps is permitted only if preceding steps fail to yield measurable improvement or satisfy the specified requirements (steps affecting parameters of negligible significance may be omitted). USB signal tuning entails inherent risks, therefore, arbitrary adjustments are strongly discouraged. It is advisable to perform parameter adjustments while concurrently monitoring the Eye Diagram during testing, with the objective of meeting compliance specifications rather than maximizing adjustment amplitude. The user assumes full and sole responsibility for all risks associated with the tuning process.

2.2 RK1808 USB2.0 Tuning Interface Description

The RK1808 USB2.0 Tuning interface is shown below. All tuning items are identical to those on the RK3568 platform. The difference lies in the fact that the USB2.0 PHY Registers on the RK1808 platform include a "write_enable" bit in the upper 16 bits, eliminating the need to read the initial Register values. For detailed descriptions of the RK1808 USB2.0 signal adjustment parameters, refer to the parameter descriptions in the subsequent RK3568 USB2.0 Tuning Interface Description section.

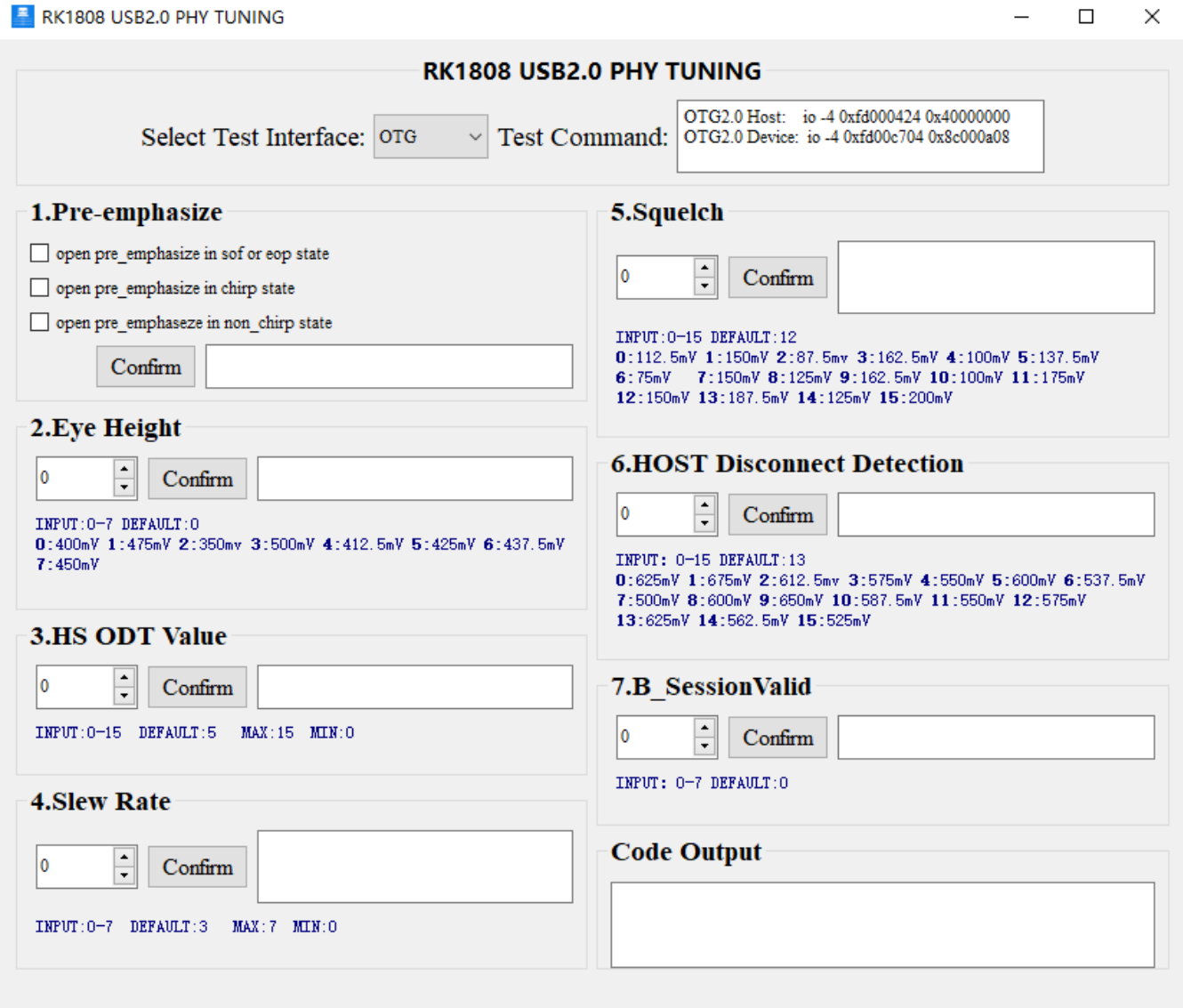


Figure 2 RK1808 USB2.0 PHY Tuning Interface

2.3 RK3399 USB2.0 Tuning Interface Description

The RK3399 USB2.0 supports four interface types: TYPE-C0, TYPE-C1, HOST0, and HOST1. The tuning process for the RK3399 consists of the following steps:

1. Select the test interface on the left side of red box 1. The test command box on the right will then automatically generate the corresponding SQ test command.
2. Configure the parameters in red box 2. The Pre-emphasis parameter uses checkboxes for selection, while other tuning items use controls similar to those in red box 4. Parameter values can be entered manually or adjusted using the up/down buttons. The input range and description for each parameter are displayed alongside its corresponding control.
3. Click the "Confirm" button in red box 3. The corresponding IO command will appear in the box on the right, and the reference code will be generated in the Code Output section (red box 5).

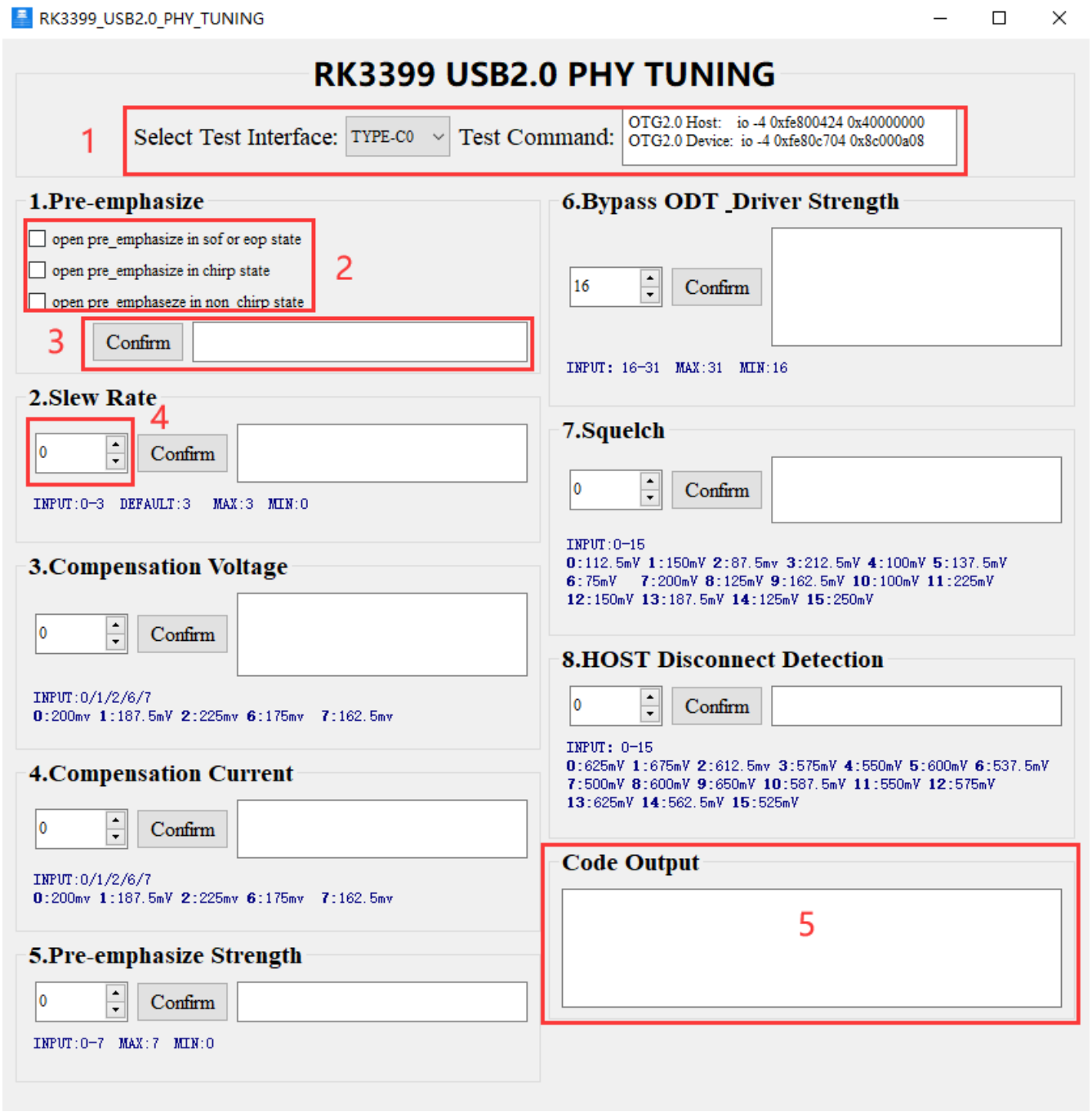


Figure 3 RK3399 USB2.0 PHY Tuning Interface

RK3399 USB2.0 Signal Tuning Parameter Description

Table 3 RK3399 USB2.0 Signal Tuning Parameters

Tuning Parameter	Description
Pre-emphasize	Adjusts HS Tx pre-emphasis
Slew Rate	Adjusts the slew rate of the HS Tx Eye Diagram
Compensation Voltage	Adjusts the HS Tx voltage calibration point, increasing this point improves the height of the USB Eye Diagram
Compensation Current	Adjusts the HS Tx current calibration point, increasing this point improves the height of the USB Eye Diagram
Pre-emphasize Strength	Adjusts the HS Tx pre-emphasis strength
Bypass ODT & Driver Strength	Bypasses the automatic resistor adjustment circuit in the comp circuit to adjust the height of the USB Eye Diagram
Squelch	Adjusts the HS Rx squelch threshold
HOST Disconnect Detection	Adjusts the HS disconnect detection threshold, used only for HOST

2.4 RK3568 USB2.0 Tuning Interface Description

The RK3568 supports four USB2.0 interfaces: OTG, HOST1, HOST2, and HOST3. The tuning process for the RK3568 consists of the following steps:

1. Select the target interface on the left side of red box 1, the corresponding SQ test command will be automatically generated in the command box on the right.
2. Since the USB PHY registers on the RK3568 platform lack the upper 16-bit "write_enable" bit, users must use the read register command provided in red box 2 to obtain the initial value of the target register. This prevents unintended modification of other unrelated bits within the same register. Enter the obtained value into the "Register Value" box on the right side of red box 2.
3. Configure the parameters in red box 3. The Pre-emphasis parameter uses a checkbox for selection, while other tuning items use controls similar to those in red box 5. Parameter values can be entered manually or adjusted using the up/down buttons. The valid input range and parameter descriptions are provided for each tuning item.
4. Click the "Confirm" button in red box 4. The corresponding IO command will appear in the box on the right, and the reference code will be generated in the Code Output section (red box 6).

Note: The register initial value box is pre-filled with a default value. If you do not manually enter the current register value, the default will be used, which may negatively affect the tuning result. Therefore, before testing, be sure to read the current register value and enter it into the Register Initial Value box.

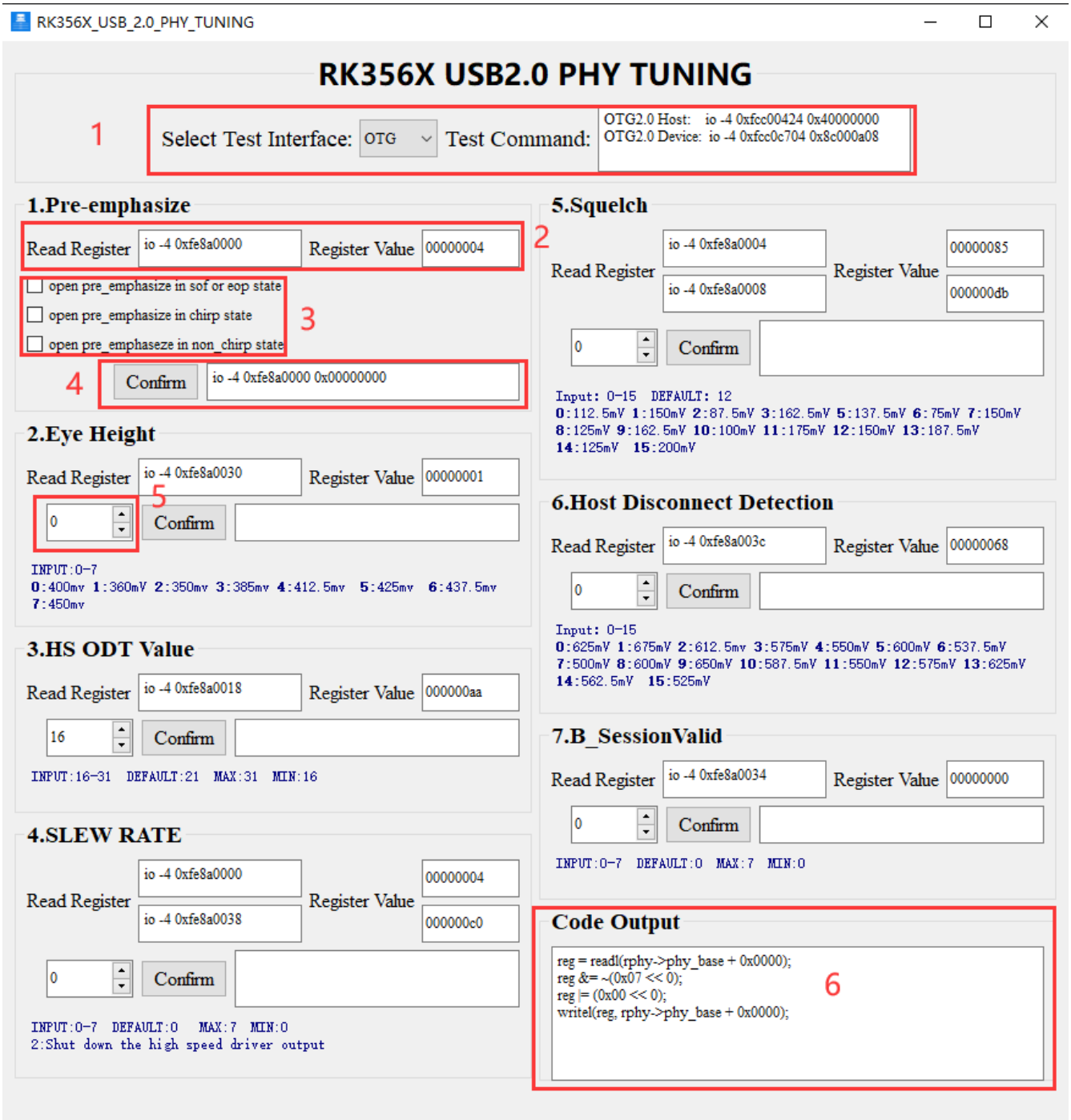


Figure 4 RK3568 USB2.0 PHY Tuning Interface

RK3568 USB2.0 Signal Tuning Parameter Overview

The RK3568 interface provides four categories of parameter tuning, including USB2.0 SQ tuning, USB2.0 noise threshold tuning, USB2.0 disconnect detection threshold tuning, and USB2.0 B_Sessionvalid tuning.

RK3568 USB2.0 SQ Tuning

USB2.0 SQ tuning primarily adjusts USB2.0 signal pre-emphasis, eye height, slew rate, and drive strength.

Table 4 RK3568 USB2.0 SQ Signal Tuning Parameters

Adjustment Parameter	Description
Pre-emphasize	Adjusts HS Tx pre-emphasis
HS Eye Height	Adjusts the height of the HS Tx Eye Diagram
HS ODT Value	Adjusts the ODT 45Ω resistor value. A larger input value results in a smaller ODT resistance and a greater Eye Diagram height, but also increases the FS/LS slew rate
Slew Rate	Adjusts the HS Eye Diagram slew rate, though the effect is minimal
Squelch	Adjusts the HS Rx squelch threshold
HOST Disconnect Detection	Adjusts the disconnect detection threshold, applicable only to HOST
B_Sessionvalid	Adjusts the VBUS DET threshold, applicable only to Device

RK3568 USB2.0 Squelch Threshold Adjustment

When a High-Speed peripheral is connected via a USB cable that is excessively long, of inferior quality, or exhibits high impedance, normal enumeration may be compromised. Under such circumstances, adjustment of the USB squelch threshold is recommended. The default squelch threshold of the USB PHY is typically set to 150mV. However, when a high-impedance cable is employed, the amplitude of legitimate USB signals may undergo substantial attenuation, potentially falling below the 150mV threshold. Consequently, valid signals risk being erroneously identified as noise by the squelch detector. To address this, the USB2.0 PHY squelch threshold may be reduced—commonly to 125mV or 112.5mV—via the Squelch Tuning mechanism.

RK3568 USB2.0 Disconnect Detection Threshold Adjustment

Adjustment of the disconnect detection threshold can be improved via Host Disconnect Detection Tuning. The logs for USB Host port peripheral identification on the Rockchip platform are as follows:

```
[ 1204.092638] usb 1-1: new high-speed USB device number 3 using ehci-platform
[ 1204.280373] usb 1-1: New USB device found, idVendor=058f, idProduct=6387, bcdDevice=
1.04
[ 1204.280449] usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[ 1204.280476] usb 1-1: Product: Mass Storage
[ 1204.280498] usb 1-1: SerialNumber: FA45A19C
```

The following logs show the disconnection of a USB peripheral from the USB Host port on the Rockchip platform:

```
[ 985.341233] usb 1-1: USB disconnect, device number 2
```

1) Unable to detect USB2.0 peripheral disconnection

In cases where the USB Host fails to detect the disconnection of a USB2.0 peripheral—indicated by the absence of disconnection logs upon removal—the following optimization steps are recommended:

1. Enable EOP pre-emphasis (EOP pre-emphasis is disabled by default in the SDK). The location to enable EOP pre-emphasis is shown in Figure 5.

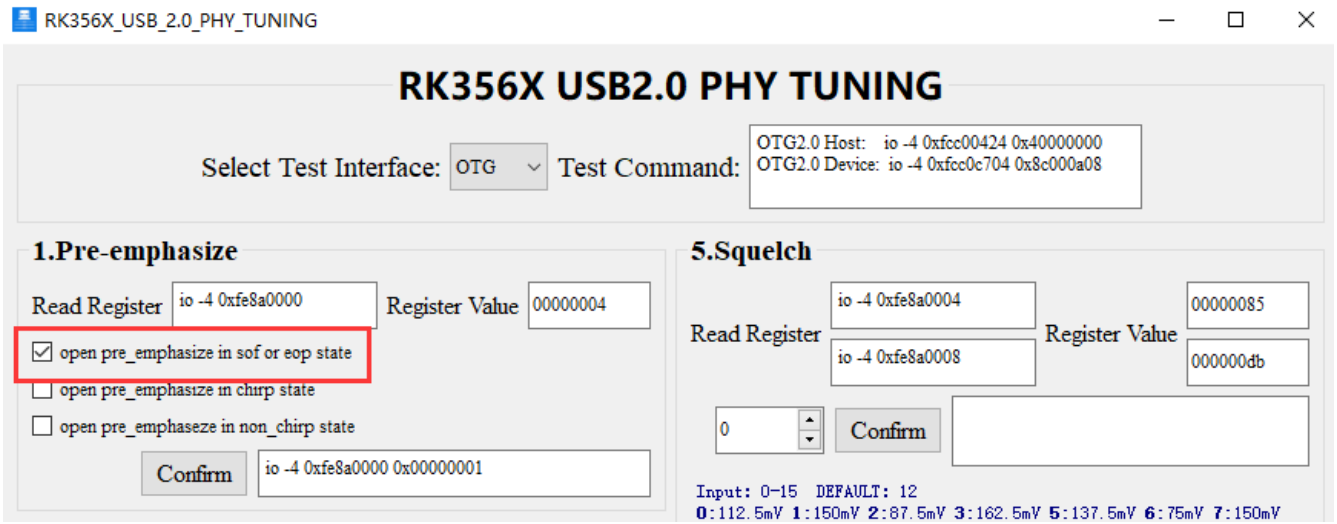


Figure 5 Enable EOP Pre-emphasis

2. Decrease the USB Host disconnect detection threshold.

Note:

- For Rockchip SoC platforms that do not support EOP pre-emphasis adjustment, Step 1 should be skipped. Please refer to the tool interface for applicable options.
- Step 2 should be performed only if Step 1 fails to yield measurable improvement.
- Reducing the disconnect detection threshold as described in Step 2 entails certain risks. Over-adjustment may result in unintended disconnection of USB peripherals during routine operation. Users are solely responsible for the evaluation and management of such risks.

2) Abnormal Disconnection of USB2.0 Peripherals

In the event that a USB2.0 peripheral experiences unexpected disconnection from the Host during normal operation, increasing the USB Host disconnect detection threshold may be considered.

Note:

- It should be noted that not all abnormal disconnections are attributable to the disconnect detection threshold. Such issues frequently arise from the USB peripheral itself or from hardware environmental factors. Accordingly, adjustment of the threshold should not be performed indiscriminately whenever such symptoms occur.
- The Rockchip SDK disables EOP pre-emphasis by default, so no Register configuration is required to disable this feature.
- Increasing the disconnect detection threshold entails inherent risks. Excessive adjustment may impair the Host's ability to detect actual disconnection events. Users assume full and sole responsibility for evaluating and managing these risks.

RK356X USB2.0 B_Sessionvalid Adjustment

B_Sessionvalid adjustment is intended to resolve issues in which a PC either fails to recognize a USB Device (e.g., ADB/MTP) or establishes recognition only to subsequently lose the connection. In such scenarios, reducing the voltage decision threshold via this parameter is recommended. The default setting is 3'b000, it is advisable to reconfigure it to 3'b111 (corresponding to option 7) or 3'b101 (corresponding to option 5).

Hardware-induced USB Device connection failures can be broadly attributed to the following three causes:

- Insufficient Vbus voltage (problems typically manifest when voltage falls below 4.7V).
- Excessive ripple on USB_AVDD1V0, or an increase of USB_AVDD1V0 to 1.2V, which prevents the PHY from correctly detecting the BValid signal.
- Excessive ripple on logic voltage supplies, leading to abnormal USB controller behavior.

2.5 RK3588 USB2.0 Tuning Interface Description

The RK3588 USB2.0 controller provides support for four interface types: TYPE-C0, TYPE-C1, HOST0, and HOST1. The tuning procedure for the RK3588 USB2.0 interface comprises the following steps:

1. Select the desired interface from the left panel of red box 1, the corresponding SQ test command will then be automatically populated in the command box on the right.
2. Configure the relevant parameters in red box 2. Parameter values may be modified either by direct manual entry or via the up/down adjustment controls. Valid input ranges and parameter descriptions are provided alongside each tuning item.
3. Click the "Confirm" button located in red box 2. The corresponding I/O command will appear in the box to the right of the button, while the reference code will be displayed in the Code Output section (red box 3).

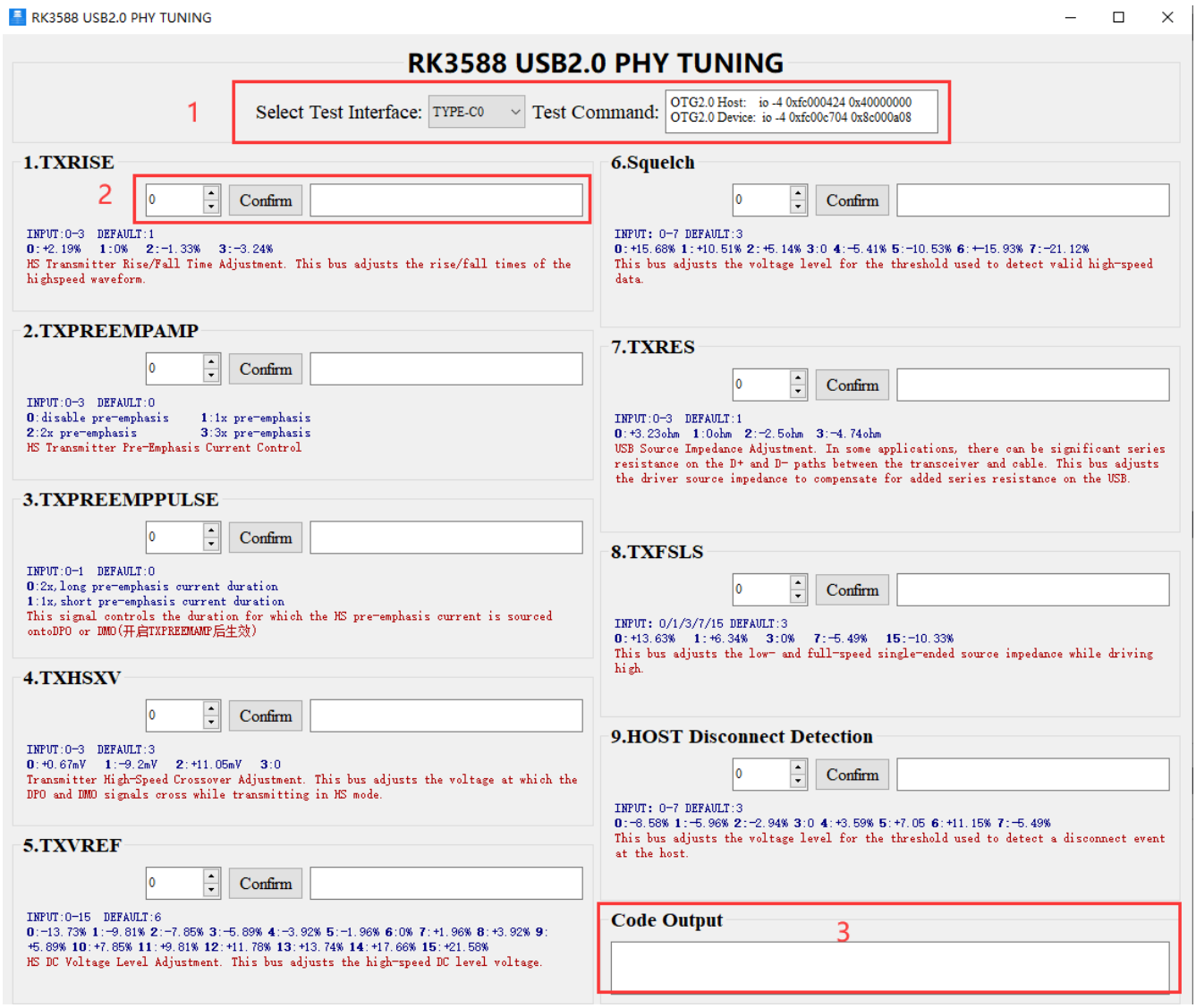


Figure 6 RK3588 USB2.0 PHY Tuning Interface

RK3588 USB2.0 Signal Tuning Parameter Description

Table 5 RK3588 USB2.0 Signal Tuning Parameters

Tuning Parameter	Description
TXRISE	Adjusts HS Tx slew rate, can modify rise and fall times
TXPREEMPAMP	Adjusts HS Tx pre-emphasis current
TXPREEMPPULSE	Adjusts HS Tx pre-emphasis current duty cycle, **this takes effect only after enabling TXPREEMPAMP TUNE**
TXHSXV	Adjusts HS Tx DP/DM crossover voltage, this adjustment shows no significant change
TXVREF	Adjusts HS Tx DC voltage, can significantly change Eye Diagram amplitude
TXRES	Adjusts HS Tx impedance, can change Eye Diagram amplitude and rise/fall times
TXFSLS	Adjusts FS/LS Tx impedance, can change Eye Diagram amplitude, rise/fall times, crossover voltage, and duty cycle
HOST DISCONNECT DETECTION	Adjusts disconnect detection threshold, used only for HOST

2.6 RV1106 USB2.0 Tuning Interface Description

The RV1106 USB2.0 Tuning interface is illustrated below. It supports adjustment of the following parameters: pre-emphasis, pre-emphasis strength, ODT, eye height, squelch threshold, Bvalid threshold, and disconnect threshold. For the RV1103 and RV1103B, Bvalid threshold adjustment is not supported due to the absence of the VBUS DET pin in the chip package, however, all other adjustment capabilities remain identical to those of the RV1106.

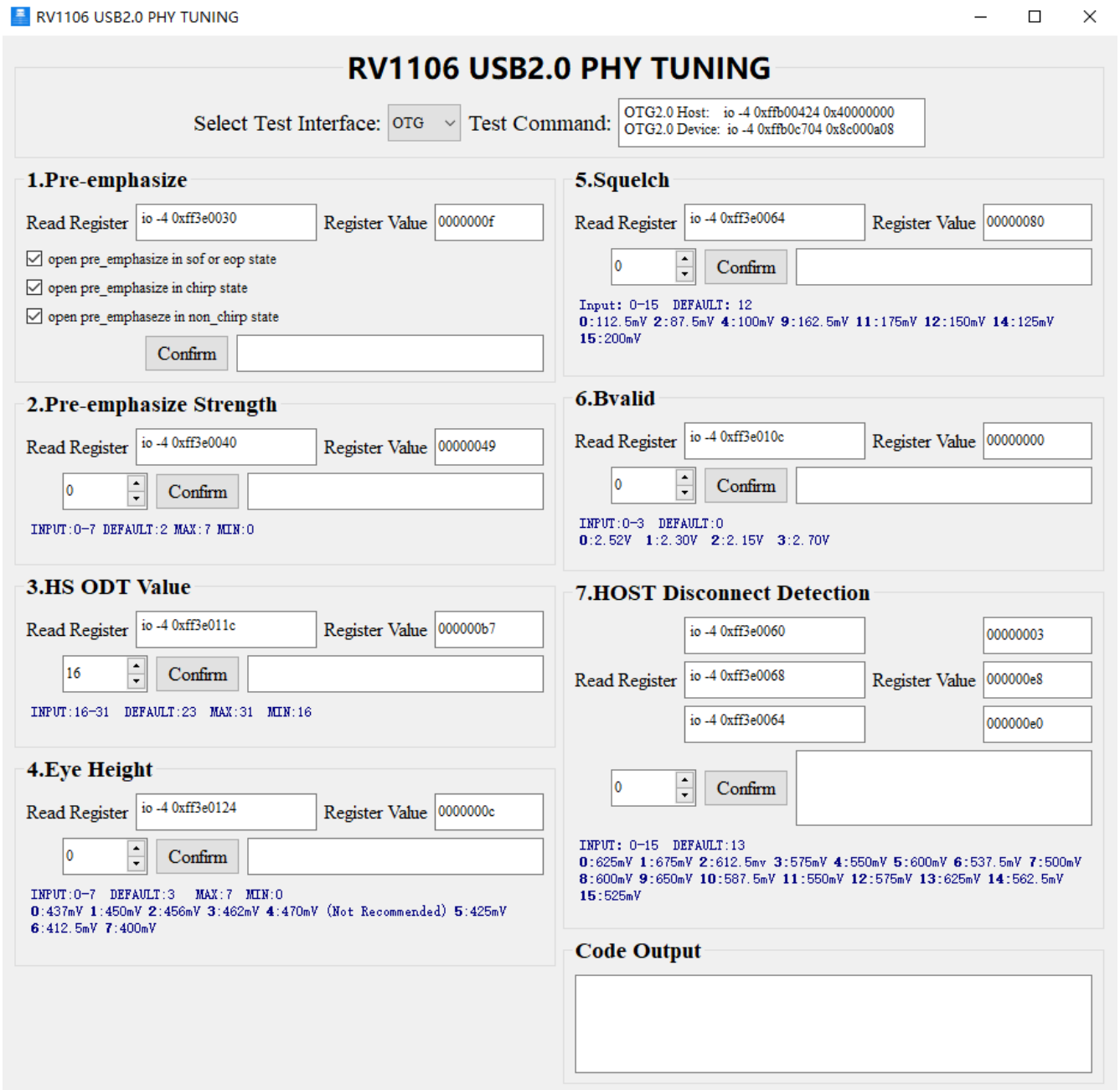


Figure 7 RV1106 USB2.0 PHY Tuning Interface

Table 6 RV1106 USB2.0 Signal Tuning Parameters

Tuning Parameter	Description
Pre-emphasize	Adjusts HS Tx pre-emphasis, enabled by default.
Pre-emphasize Strength	Adjusts HS Tx pre-emphasis strength. This Register takes effect only when pre-emphasis is enabled.
HS ODT Value	Adjusts the ODT 45Ω resistance value. A larger input value results in smaller ODT resistance and greater Eye Diagram height. This does not affect the FS/LS slew rate.
Eye Height	Adjusts the HS Tx Eye Diagram height. For level 4 (470mV), the chip logic voltage must be > 850mV.
Squelch	Adjusts the HS Rx squelch threshold.
BValid	Adjusts the VBUS DET threshold, applicable only to Device mode.
HOST Disconnect Detection	Adjusts the disconnect detection threshold, applicable only to HOST mode.

2.7 RV1109 USB2.0 Tuning Interface Description

The RV1109 USB2.0 Tuning interface is illustrated below. It supports adjustment of the following parameters: slew rate, eye height, squelch threshold and disconnect threshold. The tuning items available for the RV1126 are identical to those provided for the RV1109.

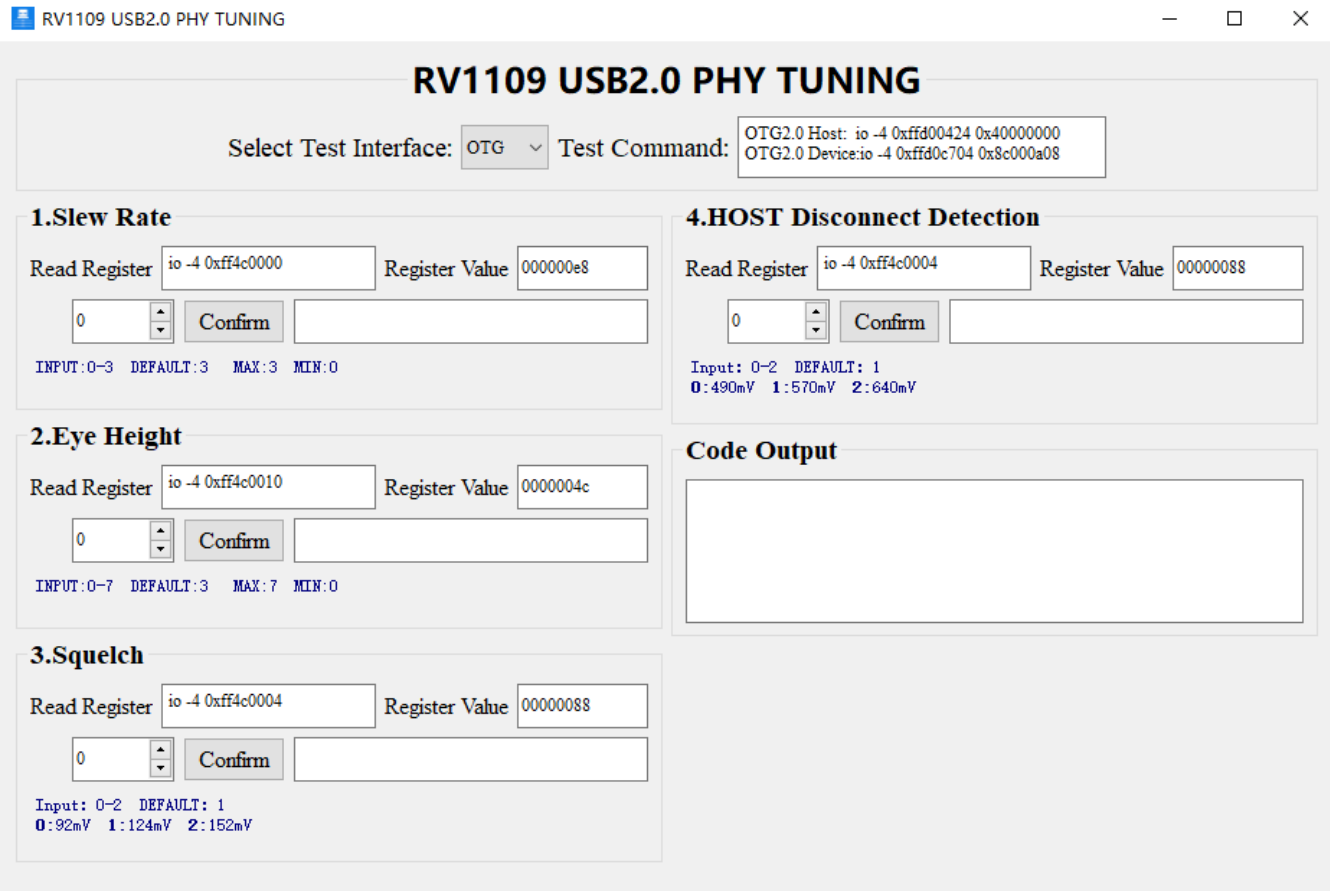


Figure 8 RV1109 USB2.0 PHY Tuning Interface

Table 7 RV1109 USB2.0 Signal Tuning Parameters

Adjustment Parameter	Description
Slew Rate	Adjusts the slew rate of the HS Tx Eye Diagram.
Eye Height	Adjusts the height of the HS Tx Eye Diagram.
Squelch	Adjusts the squelch threshold for HS Rx.
HOST Disconnect Detection	Adjusts the disconnect detection threshold, for HOST use only.

2.8 RK1808 USB 3.1 Tuning Interface Description

The RK1808 USB 3.1 Tuning interface is illustrated below. It supports adjustment of the following: Tx driver voltage, Tx driver swing, Tx driver pre-emphasis strength.

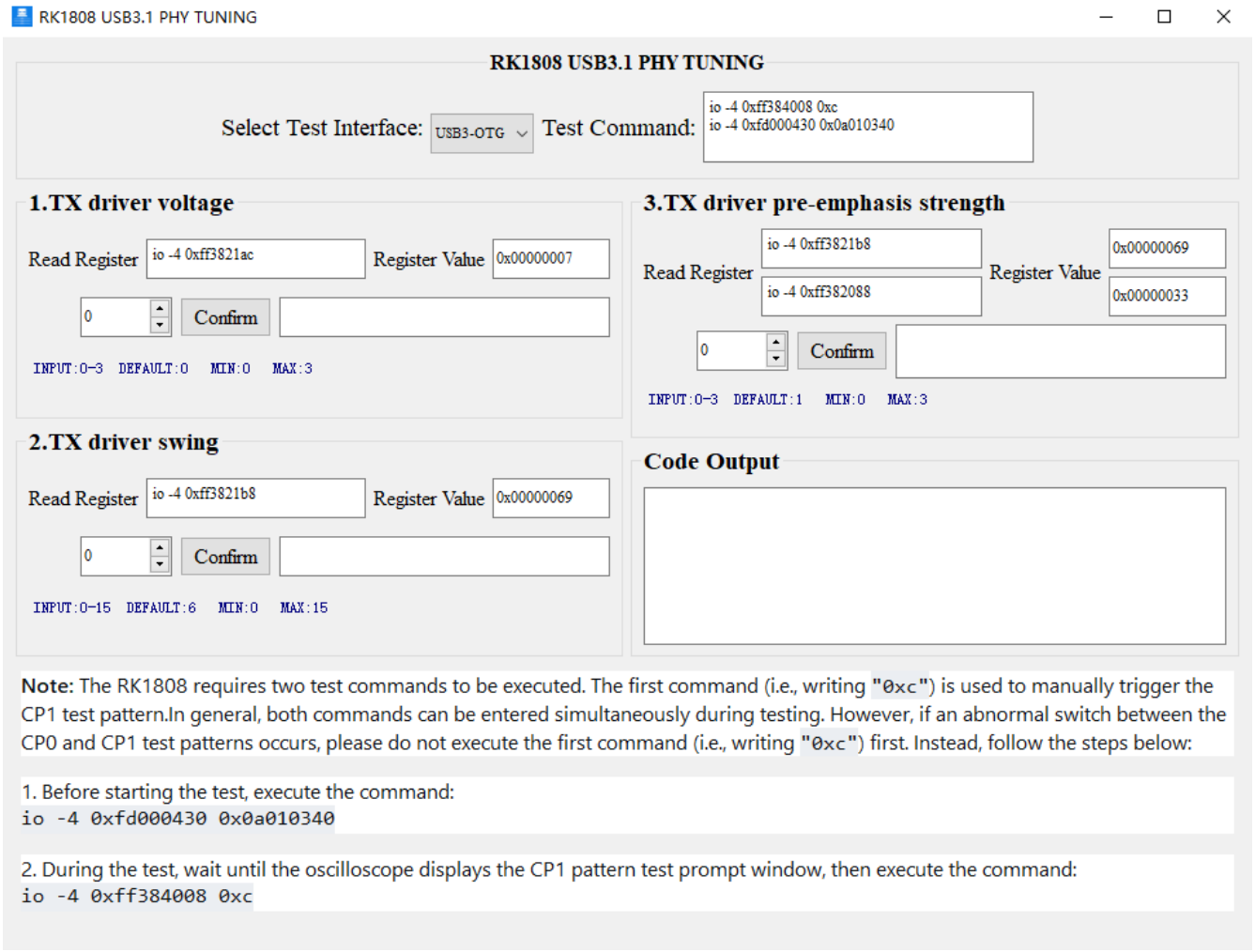


Figure 9 RK1808 USB 3.1 PHY Tuning Interface

Table 8 RK1808 USB 3.1 Signal Adjustment Parameters

Adjustment Parameter	Description
TX driver voltage	TX driver output common mode voltage tuning, tunes the common voltage of TXP/TXN differential signals
TX driver swing	TX driver swing tuning
TX driver pre-emphasis strength	TX driver pre-emphasis strength tuning

2.9 RK182X USB 3.1 Tuning Interface Description

The RK182X USB 3.1 Tuning interface is illustrated below. It supports the following parameters: Full TxSwing and TxMargin, Low Swing and TxMargin, Tx De-emphasis and Boost Level. The tuning procedure for the RK182X USB 3.1 interface comprises the following steps:

1. The RK182X USB 3.1 PHY supports PHY ID configuration. As indicated in red box 1, the register read command must first be executed to retrieve the PHY ID information. The obtained register value should then be entered into the "Enter Register Value" input field. Failure to do so will result in a prompt indicating an exception.
2. Click the "Start" button located in red box 2. This will make the "Confirm" buttons within the test items selectable.
3. Click the "Confirm" button in red box 3. The corresponding IO command will be displayed in the box to the right of the button, and the reference code will be generated in the "Code Output" section.

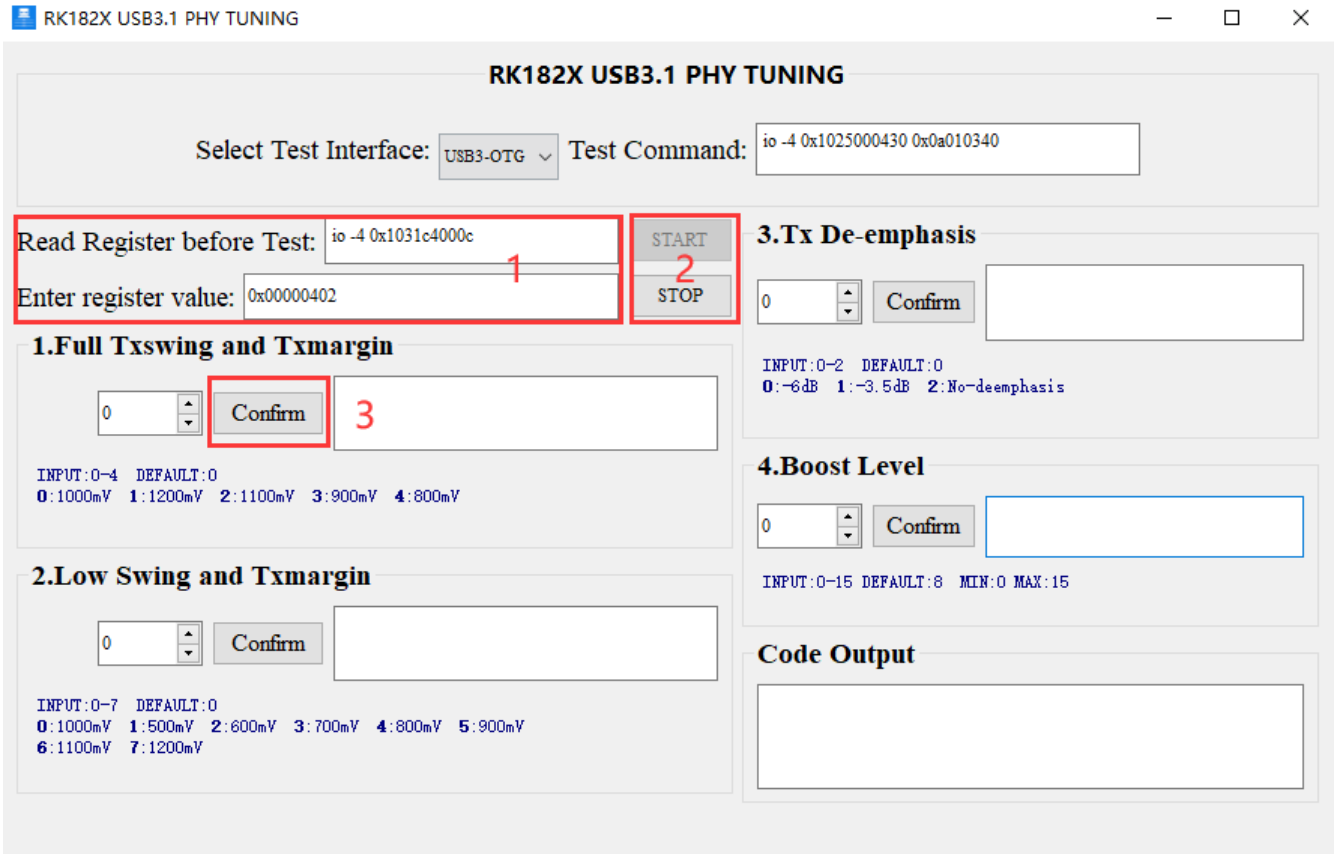


Figure 10 RK182X USB 3.1 PHY Tuning Interface

2.10 RK3399 USB 3.1 Tuning Interface Description

The RK3399 USB 3.1 controller provides support for two interface types: OTG0 and OTG1. The tuning procedure for the RK3399 USB 3.1 interface comprises the following steps:

1. Select the desired interface from the left panel of red box 1, the corresponding SQ test command will then be automatically populated in the command box on the right.
2. The RK3399 USB 3.1 PHY supports Type-C orientation detection (normal and reverse modes). As indicated in red box 2, the register read command must first be executed to retrieve the orientation information. The obtained value should then be entered into the "Enter Register Value" input field, failure to do so will trigger an exception prompt. **Note that this value must be read while the Type-C cable is properly connected.**
3. Click the "Start" button located in red box 3. At this point, the "Select Test Interface" selection in box 1 becomes locked, and the "Confirm" button in box 5 becomes enabled.

4. Since the USB 3.1 PHY registers on the RK3399 platform do not include a write_enable bit for the upper 16 bits, users must utilize the register read command provided in red box 4 to obtain the initial value of the target register. This value must be entered into the "Enter Register Value" box on the right side of box 4 to prevent unintended modification of unrelated bits within the same register.
5. Select the desired parameters in red box 5 and click the "Confirm" button. The corresponding IO command will be displayed in the box to the right of the button, and the reference code will be generated in the "Code Output" section (red box 6).

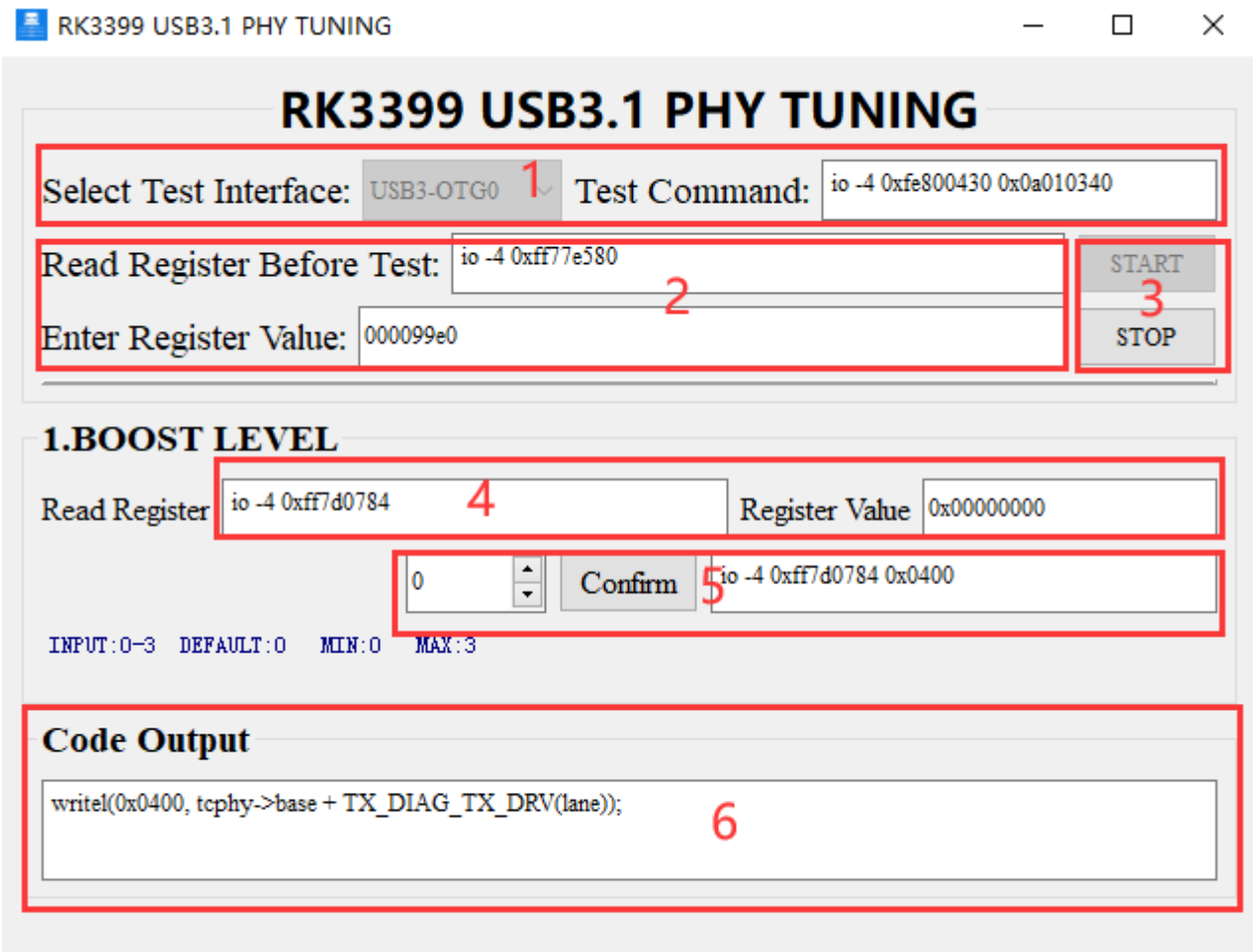


Figure 11 RK3399 USB 3.1 PHY Tuning Interface

RK3399 USB 3.1 Signal Tuning Parameter Description

Table 9 RK3399 USB 3.1 Signal Tuning Parameters

Tuning Parameter	Description
BOOST LEVEL	TX boost level adjust signal

2.11 RK3528 USB 3.1 Tuning Interface Description

The RK3528 USB 3.1 Tuning interface is illustrated below. It supports the following parameters: Full TxSwing and TxMargin, Low Swing and TxMargin, Tx De-emphasis and Boost Level.

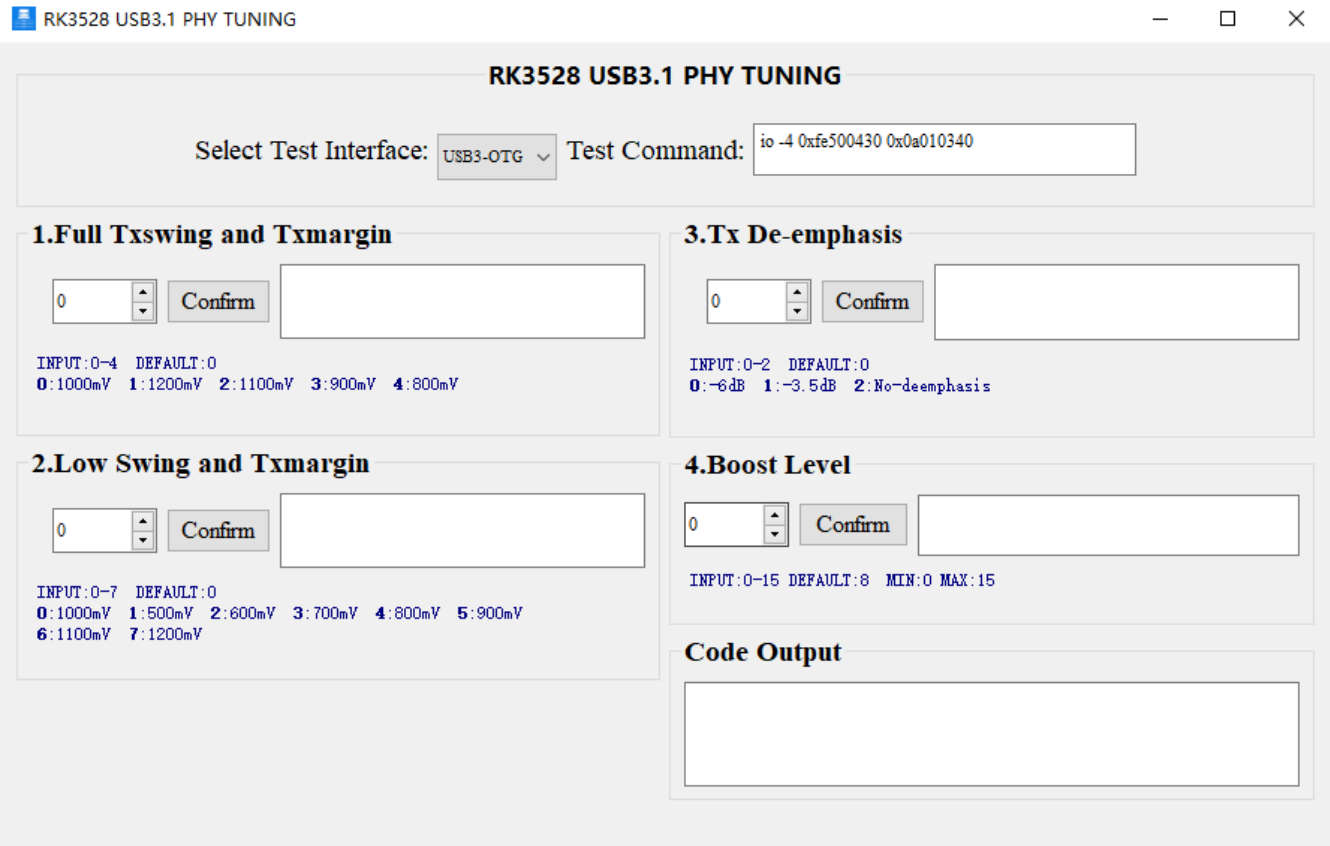


Figure 12 RK3528 USB 3.1 PHY Tuning Interface

Table 10 RK3528 USB 3.1 Signal Adjustment Parameters

Adjustment Parameter	Description
Full Txswing and TXmargin	Full swing and Txmargin Tuning. TX_SWING and TX_MARGIN[2:0] are combined together to control TX output amplitude.
Low swing and Txmargin	Low swing and Txmargin Tuning (same function as Full Tx swing, but with different gear levels)
TX De-emphasis	Transmitter de-emphasis level configuration.
Boost Level	TX boost level adjust signal

2.12 RK3588 USB 3.1 Tuning Interface Description

The RK3588 USB 3.1 controller supports tuning for three interface types: USB3-OTG0 (USB3 & DP CombPhy0), USB3-OTG1 (USB3 & DP CombPhy1) and USB3-HOST2 (USB3 & PCIE & SATA CombPhy). The tuning procedure for the RK3588 USB 3.1 interface comprises the following steps:

1. Select the desired interface from the left panel of red box 1, the corresponding SQ test command will then be automatically populated in the command box on the right.
2. When testing USB3-OTG0/1, note that CombPhy0/1 supports both normal and reverse orientation detection. As indicated in red box 2, the register read command must first be executed to retrieve the

orientation information. The obtained value should then be entered into the "Enter Register Value" field. The register value must be 0xCx or 0x3x. Otherwise, an exception will be triggered. If the register read fails—which commonly occurs with Type-C interfaces—ensure that the USB cable is properly inserted into the USB3-OTG0/1 interface under test to bring CombPhy0/1 into an operational state, then re-execute the register read command.

3. When testing the USB3-HOST ports, skip the steps described in red box 2 and proceed directly to click the "Start" button in red box 3.
4. After clicking "Start," the availability of the "Confirm" buttons is determined by the selected interface type: for USB3-OTG0/1, the "Confirm" buttons for the corresponding tuning items on the left side of the interface (red box 5) become active, while those for USB3-HOST2 on the right side (red box 6) become inactive—and vice versa.
5. Since the USB 3.1 PHY registers on the RK3588 platform lack a write_enable bit for the upper 16 bits, users must utilize the register read command provided in red box 4 to obtain the initial values of the target registers. These values must be entered into the "Register Value" field on the right side of box 4 to prevent unintended modification of other unrelated bits within the same registers.
6. Select the desired parameters in red box 5 and click the "Confirm" button. The corresponding IO command will be displayed in the box to the right of the button, and the reference code will be generated in the "Code Output" section (red box 7).

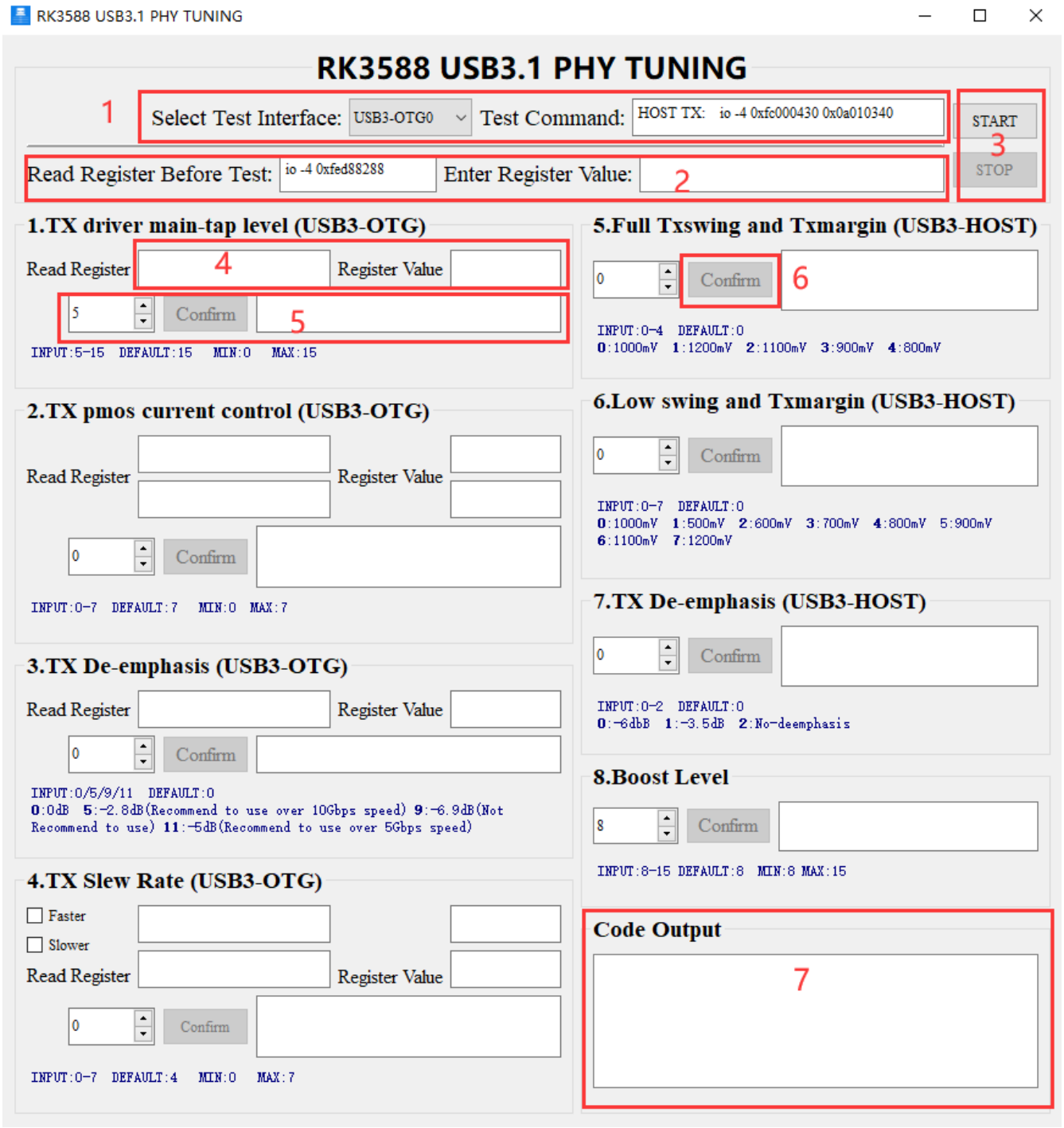


Figure 13 RK3588 USB 3.1 PHY Tuning Interface

RK3588 USB 3.1 Signal Adjustment Parameter Description

Table 11 RK3588 USB 3.1 Signal Adjustment Parameters

Parameter Adjustment	Description
TX Driver main-tap level(USB3-OTG)	Adjusts SS Tx voltage
TX Pmos current control(USB3-OTG)	Adjusts SS Tx current, recommended to configure only if voltage adjustment is insufficient
TX De-emphasis(USB3-OTG)	Adjusts SS Tx de-emphasis
TX Slew Rate(USB3-OTG)	Adjusts SS Tx Slew Rate
Full Txswing and TXmargin(USB3-HOST)	Full swing and Txmargin Tuning. TX_SWING and TX_MARGIN[2:0] are combined together to control TX output amplitude.
Low swing and Txmargin(USB3-HOST)	Low swing and Txmargin Tuning (functions identically to Full Tx swing but with different gear settings)
TX De-emphasis(USB3-HOST)	Transmitter de-emphasis level configuration.

3. Rockchip USB PHY Tuning Code Generation Instructions

3.1 RK1808 USB2.0 PHY

The RK1808 platform USB2.0 PHY Tuning code is added in the rk1808_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c.

```
static int rk1808_usb2phy_tuning(struct rockchip_usb2phy *rphy)
{
    .../* OTG/Host PHY Tuning Code */
}
```

3.2 RK182X USB2.0 PHY

The code for RK182X platform USB2.0 PHY tuning is added in the HAL_USB_HsPhyInit() function within bsp/rockchip/common/hal/lib/hal/src/hal_usb_phy.c.

```
HAL_Status HAL_USB_HsPhyInit(uint8_t id)
{
    ...
#elif defined(SOC_RK1820)
    /* OTG/Host PHY Tuning Code */
#elif defined(XXXX)
    ...
}
```

3.3 RK2116 USB2.0 PHY

The code for RK2116 platform USB2.0 PHY tuning is added in the HAL_USB_HsPhyInit() function within bsp/rockchip/common/hal/lib/hal/src/hal_usb_phy.c.

```
HAL_Status HAL_USB_HsPhyInit(uint8_t id)
{
    ...
#elif defined(RKMCU_RK2116)
    /* OTG PHY Tuning Code */
#elif defined(XXXX)
    ...
}
```

3.4 RK2118 USB2.0 PHY

The code for RK2118 platform USB2.0 PHY tuning is added in the HAL_USB_HsPhyInit() function within bsp/rockchip/common/hal/lib/hal/src/hal_usb_phy.c.

```
HAL_Status HAL_USB_HsPhyInit(uint8_t id)
{
    ...
#elif defined(RKMCU_RK2118)
    /* OTG PHY Tuning Code */
#elif defined(XXXX)
    ...
}
```

3.5 RK3036/RK312X USB2.0 PHY

The RK3036 platform currently supports only kernel version 3.10. The code is added in the usb20otg_hw_init() function within kernel/drivers/usb/dwc_otg_310/usbdev_rk3036.c.

```
static void usb20otg_hw_init(void) {
    .../* OTG/Host PHY Tuning Code */
}
```

The code for the RK312X platform is added in the rk312x_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c.

```
static int rk312x_usb2phy_tuning(struct rockchip_usb2phy *rphy)
{
    .../* OTG/Host PHY Tuning Code */
}
```

3.6 RK3228 USB2.0 PHY

The code for the RK3228 platform is added in the rk3228_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c.

```
static int rk3228_usb2phy_tuning(struct rockchip_usb2phy *rphy)
{
        .../* OTG/Host PHY Tuning Code */
}
```

3.7 RK3288 USB2.0 PHY

Since the RK3288 USB PHY Driver code has two different versions, the corresponding USB PHY modification code also differs. Therefore, when generating code on the RK3288 page, you must first select the kernel version (select 4.4 for kernel-4.4 and earlier versions, and select 4.19 for kernel-4.19 and later versions). Selecting different kernel versions only affects the format of the generated code and has no impact on the generated io commands.

RK3288 kernel-4.19 Version

Add the code at the end of the rk3288_usb_phy_probe_init() function in kernel/drivers/phy/rockchip/phy-rockchip-usb.c. Since the RK3288 has three USB ports, a USB port check must be added before the code segment converted from io commands to distinguish between different USB ports. Add the OTG interface code as follows:

```
if (rk_phy->reg_offset == 0x320) {
        ... /* OTG PHY Tuning Code */
}
```

Add the following code for the HOST1 interface:

```
if (rk_phy->reg_offset == 0x334) {
        ... /* HOST1 PHY Tuning Code */
}
```

Add the following code for the HOST2 interface:

```
if (rk_phy->reg_offset == 0x348) {
        ... /* HOST2 PHY Tuning Code */
}
```

RK3288 Kernel Version 4.4

Driver code path: kernel/drivers/usb/dwc_otg_310/usbdev_rk32.c

The RK3288 has three USB ports. The OTG port code is added at the end of the usb20otg_hw_init() function, the HOST1 port code is added at the end of the rk_ehci_hw_init() function, and the HOST2 port code is added at the end of the usb20host_hw_init() function.

3.8 RK3308B-S/RK3308 USB2.0 PHY

For the RK3308B-S platform, the code is added in the rk3308_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c, at the end of the soc_is_rk3308bs() conditional block.

```
if (soc_is_rk3308bs()) {
        ... /* OTG/HOST PHY Tuning Code */
}
```

For the RK3308 platform, add the code at the end of the else condition within the rk3308_usb2phy_tuning() function located in kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c.

```
if (soc_is_rk3308bs()) {
        ...
} else {
        .../* OTG/HOST PHY Tuning Code */
}
```

3.9 RK3326S/PX30S/RK3326/PX30/RK3328 USB2.0 PHY

The code for the RK3326S/PX30S platforms is added in the rk3328_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c, at the end of the soc_is_px30s() conditional block.

```
if (soc_is_px30s()) {
        .../* OTG/HOST PHY Tuning Code */
}
```

For the RK3326/PX30/RK3328 platforms, add the code at the end of the else conditional block within the rk3328_usb2phy_tuning() function in kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c.

```
if (soc_is_px30s()) {
        ...
} else {
        .../* OTG/HOST PHY Tuning Code */
}
```

3.10 RK3399 USB2.0 PHY

The code for RK3399 platform USB2.0 PHY tuning is added in the rk3399_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c, preceding the statement if (!of_property_read_bool(node, "rockchip,u2phy-tuning")). Since the RK3399 has four USB ports, where TYPE-C0 and HOST0 belong to one PHY group (distinguished by address: 0xe450), and TYPE-C1 and HOST1 belong to another PHY group (distinguished by address: 0xe460), a Register address check must be added before the code segment to distinguish between different USB ports.

```
if (rphy->phy_cfg->reg == 0xe450) {
        .../* TYPE-C0/HOST0 PHY Tuning Code */
} else {
        ... /* TYPE-C1/HOST1 PHY Tuning Code */
}
if (!of_property_read_bool(node, "rockchip,u2phy-tuning"))
        return ret;
```

3.11 RK3506 USB2.0 PHY

The code for RK3506 platform USB2.0 PHY tuning is added in the rk3506_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c.

```
static int rk3506_usb2phy_tuning(struct rockchip_usb2phy *rphy)
{
        .../* USB2.0 PHY Tuning Code */
}
```

3.12 RK3528 USB2.0 PHY

The code for RK3528 platform USB2.0 PHY tuning is added in the rk3528_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c.

```
static int rk3528_usb2phy_tuning(struct rockchip_usb2phy *rphy)
{
    .../* USB2.0 PHY Tuning Code */
}
```

3.13 RK3538 USB2.0 PHY

The code for RK3538 platform USB2.0 PHY tuning is added in the rk3538_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c.

```
static int rk3538_usb2phy_tuning(struct rockchip_usb2phy *rphy)
{
    .../* USB2.0 PHY Tuning Code */
}
```

3.14 RK3562 USB2.0 PHY

The code for RK3562 platform USB2.0 PHY tuning is added in the rk3562_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c.

```
static int rk3562_usb2phy_tuning(struct rockchip_usb2phy *rphy)
{
        .../* USB2.0 PHY Tuning Code */
}
```

3.15 RK3568/RK3566 USB2.0 PHY

The USB2.0 PHY tuning code for the RK3568/RK3566 platforms is shared and appended at the end of the rk3568_usb2phy_tuning() function in kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c. Since the RK3568 features four USB ports, where the OTG port and HOST1 belong to one PHY group (base address 0xfe8a0000), and HOST2 and HOST3 belong to another PHY group (base address 0xfe8b0000), the generated code segments must be prefixed with Register base address checks to distinguish between different USB ports. The code additions for the OTG interface and HOST1 interface are as follows:

```
if (rphy->phy_cfg->reg == 0xfe8a0000) {
        ... /* OTG/Host1 PHY Tuning Code */
}
```

Add the following code for the HOST2 and HOST3 ports:

```
if (rphy->phy_cfg->reg == 0xfe8b0000) {
        ... /* Host2/Host3 PHY Tuning Code */
}
```

3.16 RK3572 USB2.0 PHY

The code for RK3572 platform USB2.0 PHY tuning is added in the rk3572_usb2phy_tuning() function within the kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c file. Since the RK3572 has two USB ports, OTG0 and OTG1, the distinction between the two ports is determined by the value of rphy->phy_cfg->reg. The code insertion location is as follows:

```
static int rk3572_usb2phy_tuning(struct rockchip_usb2phy *rphy)
{
        unsigned int reg;
        int ret = 0;

        if (rphy->phy_cfg->reg == 0x0) {
                .../* OTG0 PHY Tuning Code */
        } else if (rphy->phy_cfg->reg == 0x2000) {
                .../* OTG1 PHY Tuning Code */
        }
        return ret;
}
```

3.17 RK3576 USB2.0 PHY

The code for RK3576 platform USB2.0 PHY tuning is added in the rk3576_usb2phy_tuning() function within the kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c file. Since the RK3576 has two USB ports, OTG0 and OTG1, the distinction between the two ports is determined by the value of rphy->phy_cfg->reg. The code insertion points are as follows:

```
static int rk3576_usb2phy_tuning(struct rockchip_usb2phy *rphy)
{
        unsigned int reg;
        int ret = 0;

        if (rphy->phy_cfg->reg == 0x0) {
                .../* OTG0 PHY Tuning Code */
        } else if (rphy->phy_cfg->reg == 0x2000) {
                .../* OTG1 PHY Tuning Code */
        }
        return ret;
}
```

3.18 RK3588 USB2.0 PHY

The code for RK3588 platform USB2.0 PHY tuning is added in the rk3588_usb2phy_tuning() function within the kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c file. Since the RK3588 features four USB ports (TYPE-C0, TYPE-C1, HOST0, and HOST1), these ports are distinguished based on the value of rphy->phy_cfg->reg. The code insertion point is as follows:

```
if (rphy->phy_cfg->reg == 0x0000) {
        .../* TYPE-C0 PHY Tuning Code */
} else if (rphy->phy_cfg->reg == 0x4000) {
        .../* TYPE-C1 PHY Tuning Code */
} else if (rphy->phy_cfg->reg == 0x8000) {
        .../* HOST0 PHY Tuning Code */
} else if (rphy->phy_cfg->reg == 0xc000) {
        .../* HOST1 PHY Tuning Code */
}
```

3.19 RV1103/RV1106 USB2.0 PHY

The code for the RV1103/RV1106 platform is added in the rv1106_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c.

```
static int rv1106_usb2phy_tuning(struct rockchip_usb2phy *rphy)
{
        .../* OTG PHY Tuning Code */
}
```

3.20 RV1103B USB2.0 PHY

The code for the RV1103B platform is added in the rv1103b_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c.

```
static int rv1103b_usb2phy_tuning(struct rockchip_usb2phy *rphy)
{
        .../* OTG PHY Tuning Code */
}
```

3.21 RV1109/RV1126 USB2.0 PHY

The code for the RV1109/RV1126 platform is added in the rv1126_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-naneng-usb2.c.

```
static int rv1126_usb2phy_tuning(struct rockchip_usb2phy *rphy)
{
        if (rphy->phy_cfg->reg == 0xff4c0000) {
                .../* OTG PHY Tuning Code */
        }
        if (rphy->phy_cfg->reg == 0xff4c8000) {
                .../* HOST PHY Tuning Code */
        }
}
```

3.22 RV1126B/RV1126B-P USB2.0 PHY

The code for the RV1126B/RV1126B-P platform is added in the rv1126b_usb2phy_tuning() function within kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c.

```
static int rv1126b_usb2phy_tuning(struct rockchip_usb2phy *rphy)
{
        .../* OTG/HOST PHY Tuning Code */
}
```

3.23 RK1808 USB 3.1 PHY

The code for RK1808 platform USB 3.1 PHY tuning is added in the rk1808_combphy_cfg function within kernel/drivers/phy/rockchip/phy-rockchip-inno-combphy.c.

```
static int rk1808_combphy_cfg(struct rockchip_combphy_priv *priv)
{
        ...
        if (priv->phy_type == PHY_TYPE_PCIE) {
                ...
        } else if (priv->phy_type == PHY_TYPE_USB3) {
                ...
                /*
                 * Tuning Tx:
                 * offset 0x21b8 bit[7:4]: lane 0 TX driver swing
                 * tuning bits with weight, "1111" represents the
                 * largest swing and "0000" the smallest.
                 */
                reg = readl(priv->mmio + 0x21b8);
                reg = (reg & ~0xf0) | 0xe0;
                writel(reg, priv->mmio + 0x21b8);

                .../* USB 3.1 PHY Tuning Code */


                ...
        } else {
                dev_err(priv->dev, "failed to cfg incompatible PHY type\n");
                return -EINVAL;
        }
        return 0;
}
```

3.24 RK182X USB 3.1 PHY

The code for RK182X platform USB 3.1 PHY tuning is added in the HAL_USB_SsPhyInit() function within bsp/rockchip/common/hal/lib/hal/src/hal_usb_phy.c.

```
HAL_Status HAL_USB_SsPhyInit(uint8_t id)
{
        ...
    switch (id) {
    case 0:
#ifdef COMB_PIPE_PHY0_BASE
        /* OTG PHY Tuning Code */
#endif
        break;
    case 1:
#ifdef COMB_PIPE_PHY1_BASE
        /* OTG PHY Tuning Code */
#endif
        break;
    default:
        break;
    }
        ...
}
```

3.25 RK3399 USB 3.1 PHY

The code for RK3399 platform USB 3.1 PHY tuning is added in the tcphy_tx_usb3_cfg_lane() function within kernel/drivers/phy/phy-rockchip-typec.c.

```
static void tcphy_tx_usb3_cfg_lane(struct rockchip_typec_phy *tcphy, u32 lane)
{
        .../* OTG0/OTG1 PHY Tuning Code */
}
```

3.26 RK3528/RK3562/RK3566/RK3568/RK3572 USB 3.1 PHY

The code for USB 3.1 PHY tuning on the RK3528, RK3562, RK3566, RK3568 and RK3572 platforms is added in the combphy_cfg function within the kernel/drivers/phy/rockchip/phy-rockchip-naneng-combphy.c file.

For the RK3528 platform, add the code at the following location in the rk3528_combphy_cfg function.

```
case PHY_TYPE_USB3:
        ...
        /* Enable adaptive CTLE for USB3.0 Rx */
        val = readl(priv->mmio + 0x200);
        val &= ~GENMASK(17, 17);
        val |= 0x01;
        writel(val, priv->mmio + 0x200);
        ...
        .../* USB 3.1 PHY Tuning Code */
        ...
        param_write(priv->phy_grf, &cfg->pipe_txcomp_sel, false);
```

```
        param_write(priv->phy_grf, &cfg->pipe_txelec_sel, false);
        param_write(priv->phy_grf, &cfg->usb_mode_set, true);
        break;
```

Add the RK3562 platform code at the following location in the rk3562_combphy_cfg function.

```
case PHY_TYPE_USB3:
        /* Set SSC downward spread spectrum */
        val = readl(priv->mmio + (0x1f << 2));
        val &= ~GENMASK(5, 4);
        val |= 0x01 << 4;
        writel(val, priv->mmio + 0x7c);
        ...
        .../* USB 3.1 PHY Tuning Code */
        ...
        param_write(priv->phy_grf, &cfg->pipe_sel_usb, true);
        param_write(priv->phy_grf, &cfg->pipe_txcomp_sel, false);
        param_write(priv->phy_grf, &cfg->pipe_txelec_sel, false);
        param_write(priv->phy_grf, &cfg->usb_mode_set, true);
        break;
```

The code for the RK3566 and RK3568 platforms is shared. Add the code at the following location within the rk3568_combphy_cfg function.

```
case PHY_TYPE_USB3:
        ...
        /* Set PLL KVCO to min and set PLL charge pump current to max */
        writel(0xf0, priv->mmio + (0xa << 2));
        ...
        .../* USB 3.1 PHY Tuning Code */
        ...
        param_write(priv->phy_grf, &cfg->pipe_sel_usb, true);
        param_write(priv->phy_grf, &cfg->pipe_txcomp_sel, false);
        param_write(priv->phy_grf, &cfg->pipe_txelec_sel, false);
        param_write(priv->phy_grf, &cfg->usb_mode_set, true);
```

Add the RK3572 platform code at the following location in the rk3572_combphy_cfg function.

```
case PHY_TYPE_USB3:
        ...
        /* Set PLL input clock divider 1/2 */
        rockchip_combphy_updatel(priv, GENMASK(7, 6), BIT(6), 0x14);
        ...
        .../* USB 3.1 PHY Tuning Code */
        ...
        rockchip_combphy_param_write(priv->phy_grf, &cfg->pipe_txcomp_sel, false);
        rockchip_combphy_param_write(priv->phy_grf, &cfg->pipe_txelec_sel, false);
        rockchip_combphy_param_write(priv->phy_grf, &cfg->usb_mode_set, true);
```

3.27 RK3576 USB 3.1 PHY

The PHY tuning code for RK3576 USB3-OTG0 is appended to the end of the udphy_init() function in the kernel/drivers/phy/rockchip/phy-rockchip-usbdp.c file.

```
ret = udphy_status_check(udphy);
if (ret)
        goto assert_phy;
        .../* USB3-OTG0 PHY Tuning Code */
return 0;
```

The PHY tuning code for RK3576 USB3-OTG1 is appended to the end of the rk3576_combphy_cfg() function in the kernel/drivers/phy/rockchip/phy-rockchip-naneng-combphy.c file.

```
case PHY_TYPE_USB3:
        /* Set Rx squelch input filter bandwidth */
        writel(0x0d, priv->mmio + (0x14 << 2));
        ...
        .../* USB3-OTG1 PHY Tuning Code */
        ...
        param_write(priv->phy_grf, &cfg->pipe_txcomp_sel, false);
        param_write(priv->phy_grf, &cfg->pipe_txelec_sel, false);
        param_write(priv->phy_grf, &cfg->usb_mode_set, true);
        break;
```

3.28 RK3588 USB 3.1 PHY

The code insertion points for RK3588 USB3-OTG and USB3-HOST differ, as shown below.

RK3588 USB3-OTG Code Placement

Currently, USB3-OTG lacks a dedicated interface for PHY Tuning similar to USB2.0. Therefore, the code can be appended to the end of the rk3588_udphy_init() function in the kernel/drivers/phy/rockchip/phy-rockchip-usbdp.c file.

```
ret = rk3588_udphy_status_check(udphy);
if (ret)
        goto assert_phy;
        .../* USB3-OTG PHY Tuning Code */
return 0;
```

RK3588 USB3-HOST Code Location

The USB3-HOST PHY Tuning code is added in the rk3588_combphy_cfg() function within the kernel/drivers/phy/rockchip/phy-rockchip-naneng-combphy.c file.

```
case PHY_TYPE_USB3:
        /* Set SSC downward spread spectrum */
        val = readl(priv->mmio + (0x1f << 2));
        val &= ~GENMASK(5, 4);
        val |= 0x01 << 4;
        ...
        .../* USB3-HOST PHY Tuning Code */
        ...
        param_write(priv->phy_grf, &cfg->pipe_txcomp_sel, false);
        param_write(priv->phy_grf, &cfg->pipe_txelec_sel, false);
        param_write(priv->phy_grf, &cfg->usb_mode_set, true);
        break;
```

3.29 RV1126B USB 3.1 PHY

The code for RV1126B platform USB 3.1 PHY tuning is added in the rv1126b_combphy_cfg() function within the kernel/drivers/phy/rockchip/phy-rockchip-naneng-combphy.c file.

```
case PHY_TYPE_USB3:
    /* Set Rx squelch input filter bandwidth */
    writel(0x0e, priv->mmio + (0x14 << 2));
    ...
    .../* USB 3.1 PHY Tuning Code */
    ...
    rockchip_combphy_param_write(priv->phy_grf, &cfg->pipe_sel_usb, true);
    rockchip_combphy_param_write(priv->phy_grf, &cfg->pipe_txcomp_sel, false);
    rockchip_combphy_param_write(priv->phy_grf, &cfg->pipe_txelec_sel, false);
    rockchip_combphy_param_write(priv->phy_grf, &cfg->usb_mode_set, true);
    break;
```